

Enterprise Application Infrastructure

Expanded Technical Knowledge Base - Architecture, Projects, Cloud, DevOps, Kubernetes, Data and Operations

Purpose. This document provides an end-to-end technical view of the enterprise healthcare application platform and the project delivery patterns used across its modules. It is intentionally detailed enough to explain architecture, deployment dependencies and operational behavior while remaining sanitized for safe technical learning and knowledge retrieval.

Platform scope. The application estate includes Claims, DMS, V360 UX/API, SVCS, CSVS, CEV, SSRS, Online Portal, OSM patch packaging, RecBiz Core/Query API, PHM, ED Gateway, Appian, WFM, MedDent, Correspondence Adapter and DIAAS APIs. These modules are delivered through a combination of Azure DevOps pipelines, Docker/Kubernetes, Helm, AWS services, database pipelines, reporting services and server-based components.

Infrastructure objective. Changes must be traceable from requirement to source, build, artifact, deployment and runtime. The same approved artifact should be promoted between environments wherever possible, while configuration is supplied securely for the target environment.

Layer	Primary technologies / responsibilities
Planning & source	Azure Boards, Git/Azure Repos/GitHub, branches, PR review.
Build & security	Azure Pipelines, build agents, dependency restore, Veracode, package creation.
Artifact management	JFrog Artifactory / approved image and binary repositories.
Runtime	Docker, Kubernetes/EKS, Helm, EC2/server workloads.
Data & integration	SQL Server/RDS, Redis/ElastiCache, API Gateway, SSRS.
Operations	Approvals, logs, health checks, migrations, troubleshooting, DR.

Confidentiality: no production credentials, IPs, tenant IDs, connection strings, private URLs or secret values are included. The document explains where and how such values are managed without exposing them.

Architecture Knowledge Map

Delivery -> Runtime -> Data -> Integration -> Operations

- Delivery: Boards, Git, CI/CD, security scan, artifacts
- Runtime: Docker, EKS/Kubernetes, Helm, EC2/server workloads
- Data: SQL Server/RDS, Redis/ElastiCache, SSRS
- Integration: API Gateway, internal APIs and module dependencies

1. Detailed Logical Architecture

The platform follows a layered enterprise architecture. Work management and source control initiate the delivery flow. CI creates validated artifacts. CD promotes those artifacts to runtime targets. Runtime services depend on data, cache and integration layers, while security and observability apply across the complete path.

```
DELIVERY PLANE Work Item -> Git -> PR -> CI Build -> Security Scan -> Artifact Repository | v Release / Approval | v
DEPLOYMENT PLANE Azure DevOps CD Pipelines / | \ / | \ v v v Helm/EKS DB Pipeline Server/EC2 | | |
+-----+-----+ | v APPLICATION PLANE UX / APIs / Services / Batch / Workflow | | | v v v API Gateway
Redis SQL/RDS/SSRS | v Consumers / Integrations
```

The **delivery plane** is responsible for software provenance. It answers: which requirement caused the change, which commit contains it, which build created the artifact, and which scan validated it. The **deployment plane** determines where that artifact runs and injects the correct environment configuration.

The **application plane** contains user interfaces, APIs, backend services, batch jobs, workflow components and reporting functions. These workloads may be containerized or server-based depending on module design. The **data and integration plane** supplies databases, cache, API routing and reporting connectivity.

A failure is isolated by identifying the layer first. Build failures are investigated in CI and agent dependencies; configuration failures in CD/variables; pod failures in Kubernetes; data failures in SQL/RDS; request-routing failures in gateway/service networking.

Layer Interaction Architecture

Source -> CI -> Artifact -> CD -> Runtime -> Data/Integration

- Control plane: work item, branch, PR, approvals and release evidence
- Application plane: UX, APIs, services, workflow and batch jobs
- Dependency plane: database, cache, gateway and downstream modules
- Failure isolation starts by identifying the affected architecture layer

2. Runtime Request and Data Flow

A typical application request enters through a controlled API or application endpoint, reaches the appropriate service, and may call cache, database or downstream integration components. The exact route differs by module, but the troubleshooting principle remains consistent.

```
Client / UX | v API Gateway / Entry Point | v Kubernetes Service / Application Endpoint | v Application Pod / API /  
Backend Service | +-----> Redis / ElastiCache | +-----> SQL Server / RDS | +-----> Internal Module API | `----->  
External / Enterprise Integration Response returns through the same controlled service path.
```

V360 example. V360 UX represents the user-facing layer while V360 API provides backend capability. The UX/API release depends on valid environment configuration. If required token values are missing, deployment should stop before the application is considered ready. Once deployed, service health and API connectivity are validated.

RecBiz Core / Query API example. These API workloads require pipeline definitions, variable groups and environment-specific values. Their delivery is not complete until required configuration ownership - such as tenant or portal-related values - has been resolved and validated.

Gateway example. ED Gateway/API Gateway sits at the integration boundary. A failed API call may be caused by route configuration, a downstream Kubernetes service, unhealthy pods, DNS/TLS, database connectivity or a dependent module. Operations therefore traces the complete request rather than assuming the gateway itself is the root cause.

Cache behavior. Redis/ElastiCache provides low-latency access for cache/session patterns. Applications should tolerate expected cache behavior and expose useful errors when the cache is unreachable rather than hiding dependency failures.

Request Processing Architecture

Client/UX -> Gateway -> Service -> Pod/API -> Cache/DB/Downstream API

- Gateway validates/routs traffic to the correct application endpoint
- Backend services read/write persistent data and use cache where applicable
- Kubernetes Service provides stable access to healthy application pods
- Response health depends on every downstream dependency in the request path

3. Environment Topology and Release Promotion

The estate uses multiple environments so changes can be validated progressively. Environment isolation applies to application configuration, deployment targets, database endpoints, cache endpoints, access permissions and sometimes infrastructure capacity.

Environment	Role	Deployment characteristics
Development	Early technical integration.	Frequent deployment; configuration and dummy/smoke scripts may be validated.
Test / CST	Functional/integration verification.	Controlled packages, module testing and database/API validation.
Hotfix	Urgent correction against release baseline.	Dedicated HF branches/pipelines; minimal scoped change.
Pre-Production	Production-like verification.	Release candidate, stricter approval and realistic configuration.
Production	Live workload.	Restricted deployment access, formal approval, evidence and recovery planning.
DR	Business recovery target.	Compatible release package/configuration synchronized per recovery process.

Promotion should reuse the same artifact rather than rebuilding separately in every environment. This reduces the possibility that Test validates one binary while Production receives another. Differences belong in protected environment configuration.

Release branches identify stable baselines. Deploy branches can represent the exact delivery line used by pipelines. Hotfix branches isolate emergency corrections. When branch conventions change, pipeline triggers and package logic must be updated together to prevent incorrect builds.

For server migrations, infrastructure details change while application intent remains the same. New servers or targets require updated tokens/groups, pipeline targets, connectivity and prerequisite validation before deployment is enabled.

Environment Architecture

Dev -> Test/CST -> Hotfix/Pre-Prod -> Production -> DR

- The approved artifact is promoted rather than rebuilt per environment
- Variable groups and protected values supply environment-specific settings
- Production adds stricter approval, evidence and recovery requirements
- DR must retain a compatible application, database and configuration set

4. Azure DevOps CI/CD Architecture

Azure Boards provides the work-item entry point. A user story or defect defines the required change and acceptance criteria. A source branch implements the change. A pull request provides review and validation before the controlled branch is updated.

```
User Story / Bug | Feature / Fix Branch | Pull Request + Review | Merge to approved branch | CI: Restore -> Build -> Validate -> Scan -> Package | Publish immutable artifact | CD: Load config -> Deploy DB -> Deploy App -> Validate | Environment approval / promotion | Deployment evidence linked to change
```

CI pipelines should fail fast when required dependencies are unavailable. For example, service builds can fail when a native library such as a shared Linux library is missing. The correct resolution is to restore the dependency in the build environment or package definition, not manually alter a production server.

CD pipelines load variable groups and protected configuration. The deployment logic can perform token replacement, select a Helm values file, run database scripts, deploy binaries, or target a server/deployment group depending on module type.

A release is complete only after validation. Useful validation includes pipeline status, application health, pod readiness, API smoke tests, database script status and confirmation from the responsible module team where functional verification is required.

Pipeline concern	Control
Branch/version mismatch	Validate source branch and artifact version before deployment.
Missing token	Stop pipeline and obtain approved environment value.
Agent failure	Check agent health, capability and network access.
Package unavailable	Verify repository path/version and publication result.
Approval pending	Do not bypass controlled environment approval.

CI/CD Control Architecture

Board -> Branch -> PR -> CI -> Scan -> Artifact -> CD -> Validate

- CI owns compilation, tests, scanning, packaging and immutable versioning
- CD owns target configuration, database/app deployment and health checks
- Service connections provide controlled access to deployment destinations
- A failed prerequisite stops promotion before partial deployment occurs

5. Artifact, Package and Patch Management

JFrog Artifactory or an approved repository is the system of record for build outputs. It separates the act of building software from the act of deploying it. This improves reproducibility and makes rollback to a known version possible.

Artifacts can include application binaries, DLLs, container images, database scripts, Helm charts, report packages and supporting configuration templates. Each artifact should carry enough metadata to identify its module, release, build and source commit.

OSM patch packaging. Cross-module patch preparation depends on each module supplying its required release item. V360, WFM, DMS, MedDent or Correspondence Adapter items may progress at different times, so the final package must explicitly track missing dependencies rather than silently omit them.

Hotfix packaging. A hotfix package should be distinguishable from a standard monthly release. It should include only approved changes required for the fix and retain a clear relationship to the baseline it corrects.

Package content	Examples / purpose
Application binary	Compiled DLL/JAR/package or other runtime binary.
Container image	Immutable image for Kubernetes deployment.
DB scripts	Schema/data changes required by the release.
Helm chart/values	Kubernetes workload definition and environment parameters.
Reporting package	SSRS reports/data-source definitions where applicable.
Release metadata	Build ID, commit, version, branch, checksum, scan result.

Mutable labels such as 'latest' are poor production evidence. The deployment record should identify a specific immutable artifact or image tag.

Artifact Promotion Architecture

Build Output -> Repository -> Release Package -> Environment Deployment

- Artifacts are immutable and tied to build ID, commit and release version
- Cross-module patches track every required package before final assembly
- Hotfix packages remain traceable to the baseline they correct
- Rollback uses a known artifact, not a locally rebuilt replacement

6. Kubernetes/EKS Runtime Architecture

Amazon EKS provides managed Kubernetes capability for containerized workloads. The cluster runs application pods while Kubernetes objects define desired state, networking and health behavior. Environment separation can use namespaces and infrastructure boundaries according to platform design.

Object	Role in project runtime
Deployment	Defines application image, replicas, rollout strategy and pod template.
Pod	Runs one or more containers for an application instance.
Service	Provides stable discovery/network access to changing pod instances.
ConfigMap	Holds non-secret configuration supplied to workloads.
Secret / external reference	Supplies protected runtime configuration.
Job / CronJob	Supports batch or scheduled workloads.
Readiness probe	Determines when a pod may receive traffic.
Liveness probe	Detects an unhealthy process that should be restarted.

During deployment, Kubernetes pulls the approved image, creates new pods and waits for readiness. A healthy rollout requires both Kubernetes state and application behavior to be correct. A pod can be Running but still fail readiness because a database, Redis endpoint or dependent API is unavailable.

Typical investigation commands or platform views focus on deployment rollout, pod state, events and logs. Operations should compare the image tag actually running with the artifact expected from the release pipeline.

Resource requests and limits protect cluster stability. Under-sized resources can cause throttling or termination; excessive requests reduce scheduling efficiency. Runtime capacity should be tuned using observed workload behavior rather than arbitrary values.

EKS Workload Architecture

Helm Release -> Deployment -> ReplicaSet -> Pods -> Service -> Consumers

- ConfigMaps provide non-secret configuration; secrets use protected references
- Readiness controls traffic admission; liveness detects unhealthy processes
- Services decouple stable addressing from changing pod instances
- Logs, events, probes and image tags provide the main runtime evidence

7. Helm and Configuration Architecture

Helm provides a reusable packaging layer above Kubernetes YAML. A chart describes the common workload structure while values provide deployment-specific settings. This supports consistent deployments across Dev, Test, Hotfix and Production.

```
application-chart/ |-- Chart.yaml # chart identity/version |-- values.yaml # safe defaults `-- templates/ |--
deployment.yaml # image, replicas, probes, resources |-- service.yaml # stable application network endpoint |--
configmap.yaml # non-secret configuration `-- optional resources # jobs, ingress, service accounts CD pipeline ->
selects chart + approved image tag -> loads environment values -> resolves protected secrets -> helm upgrade/install
-> validates rollout
```

Environment values should define only the settings that legitimately differ, such as replica count, endpoint references or resource sizing. Secrets should be injected through protected mechanisms rather than committed to chart values.

Helm history helps identify which release revision changed the workload. When a deployment fails, compare the current and previous revision, image tag, rendered configuration and pod events. Rollback is appropriate only when the previous application version remains compatible with database and integration state.

V360 UX/API and other Kubernetes-hosted modules benefit from this pattern because the same deployment structure can be reused while environment values remain controlled outside the application binary.

Helm Release Architecture

Chart + Image Tag + Environment Values -> Helm Upgrade -> Kubernetes

- Chart templates define reusable deployment/service/configuration resources
- Environment values change configuration without rebuilding the application
- Helm history records revisions and supports controlled release comparison
- Rollback requires compatibility with current database and integration state

8. AWS, Database, Cache and Reporting Architecture

Service	Platform responsibility	Typical dependency
EC2	VM compute for utilities/server workloads.	OS, network, service process, deployment target.
EKS	Container orchestration.	Images, pods, services, Helm and cluster access.
RDS / SQL Server	Persistent relational data.	Connection, schema, permissions and query health.
ElastiCache / Redis	Low-latency cache/session layer.	Endpoint, auth, memory and network path.
API Gateway	Controlled API routing/exposure.	Routes, integration targets and downstream health.
SSRS	Enterprise reporting.	Report package, data source, DB access and permissions.

Database pipeline. Database scripts are versioned alongside release work and deployed through controlled automation. The pipeline verifies the target, obtains protected connection information, executes scripts in order, captures errors and validates the expected schema/data state.

Compatibility. Application and database versions must remain compatible. If a release requires a schema change before a new API can start, the pipeline sequence must enforce that dependency. High-risk transformations require recovery planning before production.

SSRS. Reporting deployment is more than copying a report file. Reports can depend on shared data sources, database permissions and environment-specific references. Validation should confirm both successful publication and correct data retrieval.

Redis. Cache health should be considered when troubleshooting intermittent application behavior. A cache endpoint can be reachable while authentication, memory pressure or application key behavior still causes failures.

Data and Integration Architecture

Application -> Redis Cache -> SQL/RDS | Gateway -> Internal/External APIs

- RDS/SQL Server stores persistent transactional data
- Redis/ElastiCache provides low-latency cache or session behavior
- SSRS consumes approved data sources for enterprise reporting
- API Gateway separates consumer-facing routes from downstream implementation

9. Project Module Architecture and Responsibilities

Module	Technical role / deployment pattern
Claims	Core application workload with binary, database and API-related release changes. Hotfixes require controlled baseline packaging.
DMS	Document-management service; binary/core DLL changes are packaged and promoted through environment pipelines.
V360 UX	User-facing component; deployed with environment-specific configuration and aligned to compatible API versions.
V360 API	Backend API services; Kubernetes/Helm and token/configuration validation are important deployment dependencies.
SVCS / CSVS / CEV	Backend/service modules. Build dependency availability can block CI; releases follow controlled artifact promotion.
SSRS	Reporting layer tied to database/data-source configuration and report-server deployment.
Online Portal	Portal workload with platform engineering and environment integration dependencies.
RecBiz Core / Query API	API services requiring CI/CD, variable groups, protected tokens and tenant/portal configuration.
PHM	Application/integration reference area used for compatible configuration patterns.
ED Gateway	Gateway/integration component controlling connectivity between API consumers and downstream services.
Appian / WFM	Workflow/business-process platform components with package and environment deployment requirements.
MedDent	Module with release/package dependencies that may participate in hotfix or patch packaging.
Correspondence Adapter	Integration/adaptor package that can be a dependency in cross-module releases.
DIAAS APIs	API workloads with CD setup and Azure DevOps environment variables.

The modules do not operate as isolated deployment units. Shared databases, gateways, configuration values and release packages create dependencies. Release planning therefore needs a module matrix showing which component changes, what artifact is required, whether DB work exists, and who validates the deployment.

Module Dependency Architecture

UX -> API -> Shared Services -> Database/Cache -> Gateway/Integrations

- V360 UX and API must remain configuration and version compatible
- Claims/DMS/service modules may include binary and database dependencies
- RecBiz APIs depend on protected environment and tenant/portal configuration
- Patch packaging coordinates module versions so dependent releases stay aligned

10. Representative Project Deployment Scenarios

V360 database pipeline setup. A DB pipeline is created for target environments, variable groups are configured, and a harmless/dummy script can be used to verify connectivity and pipeline behavior before real release scripts are introduced. After successful validation, the pipeline code is reviewed and merged.

Hotfix deployment. A hotfix branch is aligned to the required release baseline. CI produces the hotfix artifact. CD loads hotfix environment values and deploys only the approved module changes. After technical validation, the package can progress through the required approval path.

DMS binary/Core DLL release. Updated binaries are built or collected into the approved package structure, versioned, and promoted through the DMS deployment path. Package completeness and dependency versions are checked before release.

RecBiz Core/Query API onboarding. Pipeline definitions and variable groups are created. Reference configuration patterns can be taken from a compatible module, but module-specific values must come from the responsible owner. Deployment remains pending until required tenant/portal configuration and approvals are available.

Server migration. When old servers are decommissioned and new servers are created, pipelines are not simply pointed at the new host. Deployment targets/groups, tokens, connectivity, permissions and prerequisites are updated and validated. Only then are pipelines enabled for deployment.

Batch/API deployment. Batch jobs and APIs can have different execution patterns but should share the same release controls: source traceability, immutable artifacts, environment configuration, deployment logs and post-deployment validation.

These scenarios demonstrate a consistent operating model: dependencies are resolved before promotion, environment-specific values are controlled, and deployment success is proven rather than assumed.

Deployment Scenario Architecture

Prerequisite -> Pipeline Setup -> Configuration -> Deploy -> Validate -> Merge/Close

- New pipelines are first validated with safe test/dummy deployment content
- Module-owned values are obtained from the responsible team before promotion
- Server migrations update targets, groups, permissions and connectivity
- A scenario closes only after technical and module validation are complete

11. Security and Access Architecture

Security is layered across the software delivery lifecycle. Source repositories protect changes through branch policies and pull requests. CI performs security checks. Artifact repositories control what binaries are available. CD service connections limit deployment permissions. Runtime platforms restrict workload and data access.

Boundary	Controls
Source	Repository permissions, PR review, branch protection.
Build	Trusted agents, dependency control, security scanning, build logs.
Artifact	Approved repository, immutable version, provenance/checksum.
Deployment	Protected variables, secret references, service connections, approvals.
Kubernetes	RBAC/namespaces, protected secrets, service identity, network controls.
Database	Restricted credentials, private connectivity, least-privilege permissions.
Documentation	No credentials, private URLs, keys, connection strings or sensitive IDs.

Veracode or similar application security scanning should be associated with the exact build being promoted. A scan result for a different commit or package does not provide valid evidence for the current release.

Secrets are not application configuration in the ordinary sense. They require protected storage, restricted access and careful logging. Pipelines should avoid echoing secret values. Documentation should describe a placeholder or secret reference rather than the underlying value.

Production access should be narrower than lower environments. Emergency or temporary access should follow organizational approval and removal procedures. Shared personal credentials should never be used as a workaround for access problems.

Security Boundary Architecture

Developer -> Repo Policy -> CI Scan -> Artifact Trust -> CD Identity -> Runtime RBAC

- Least privilege applies to repositories, pipelines, cloud and databases
- Secrets remain in protected stores/variables and are not printed to logs
- Production changes require authorized approvals and traceable evidence
- Security results must correspond to the exact artifact being promoted

12. Monitoring, Troubleshooting and Incident Isolation

Operational troubleshooting is most effective when it separates delivery failures from runtime failures. First identify whether the change built correctly, deployed correctly, started correctly and can reach its dependencies.

Symptom	Investigation path
Build fails	Agent -> checkout -> dependency restore -> compiler/package -> scan/build logs.
CD fails before deployment	Artifact -> variables -> token replacement -> service connection -> target access.
Pod not starting	Image/tag -> registry pull -> config/secret -> container logs -> resource limits.
Pod running but not ready	Readiness probe -> DB/Redis/API dependencies -> application logs.
API unavailable	Gateway -> route -> service -> pod -> downstream data/integration.
DB deployment fails	Target -> credentials/reference -> connectivity -> script order -> SQL error.
Wrong version	Work item -> branch -> commit -> build -> artifact -> Helm/package -> runtime.
Migration issue	New target -> network -> permissions -> deployment group -> variables -> prerequisites.

Pipeline logs explain delivery behavior, Kubernetes events explain scheduling/runtime state, application logs explain service behavior, and database/gateway logs explain dependency behavior. Correlating timestamps and artifact versions across these sources prevents unrelated errors from being treated as one incident.

When a missing token causes deployment failure, the correct action is to obtain the approved value from the responsible owner and update protected configuration. When a module team must provide a connection string or endpoint, the infrastructure team should not invent a replacement value.

A deployment can be technically successful but functionally incomplete. Module-team validation or a smoke test should confirm that the business-facing function works after infrastructure health checks pass.

Observability Architecture

Pipeline Logs -> Platform Events -> App Logs -> Dependency Logs -> Root Cause

- Pipeline evidence answers what was built and deployed
- Kubernetes events answer scheduling, image and probe problems
- Application logs expose startup, configuration and request failures
- Gateway/database/cache evidence confirms downstream dependency health

13. Reliability, Recovery and Operational Ownership

Reliable enterprise delivery assumes that failures will occur and designs recovery into the process. Immutable artifacts, deployment history, database recovery planning and DR package synchronization reduce recovery time.

Application rollback. A previous application artifact can be redeployed only when it remains compatible with current database and integration state. Kubernetes/Helm history helps identify the previous runtime revision, but rollback decisions must consider data changes.

Database recovery. Schema/data changes can be harder to reverse than application binaries. Production database changes therefore require pre-deployment checks, backups where required, and an agreed rollback/restore/forward-fix strategy.

DR readiness. Disaster-recovery locations should receive compatible application, database and configuration packages according to the recovery plan. Missing DR artifacts are a release-readiness risk.

Responsibility	Typical owner interaction
Application code/package	Module/development team supplies and validates functional change.
CI/CD pipeline	DevOps maintains pipeline logic, variables and deployment automation.
Cloud/server platform	SRE/Infrastructure provides runtime capacity, network and server prerequisites.
Database	DB team validates database connectivity/change execution where required.
Security	Security tooling/policy controls scanning and release findings.
Release approval	Authorized leads/owners approve controlled environment promotion.

Clear ownership prevents stalled deployments. If a value belongs to a module team, if an approval belongs to a release owner, or if a server prerequisite belongs to infrastructure, the pipeline status and release communication should make that dependency visible.

Recovery Architecture

Known Good Artifact + Compatible Data + Configuration + DR Package -> Recovery

- Application rollback is safe only when data/integration compatibility remains
- Database recovery may require restore or forward-fix rather than simple rollback
- DR readiness includes package, configuration and dependency synchronization
- Clear ownership reduces recovery delay across DevOps, SRE, DB and module teams

14. Complete End-to-End Project Lifecycle

The full infrastructure lifecycle begins before a pipeline runs. A work item defines the objective, dependencies and acceptance criteria. Development implements the change on the correct branch. Pull-request review confirms code quality and alignment to the release baseline.

CI then creates a traceable artifact. The artifact is scanned, versioned and stored. CD selects that exact artifact and combines it with target-environment configuration. If database work exists, the release sequence enforces the required dependency. Kubernetes/Helm or server deployment then places the application into runtime.

Post-deployment checks prove that the service is healthy, not merely that the pipeline completed. This includes pod readiness or process health, API response, database/cache connectivity and module-specific validation. Production promotion requires the appropriate approval and evidence.

REQUIREMENT | SOURCE CHANGE | PULL REQUEST | CI BUILD + SECURITY | VERSIONED ARTIFACT | ENVIRONMENT CONFIGURATION | DB / APP / HELM DEPLOYMENT | TECHNICAL HEALTH CHECK | MODULE / FUNCTIONAL VALIDATION | APPROVAL AND PROMOTION | PRODUCTION + MONITORING + RECOVERY EVIDENCE

The most important architecture principle across all projects is separation of concerns: source defines the change; CI proves and packages it; the artifact repository preserves it; CD configures and deploys it; runtime platforms execute it; data/integration services support it; and monitoring plus release evidence prove what is operating.

Final operational standard: every release should be reproducible, traceable, securely configured, dependency-aware, observable and recoverable. This standard applies whether the workload is a Kubernetes API, UX component, database script, SSRS report, batch job, workflow package or server-hosted service.

Lifecycle Architecture Summary

Requirement -> Source -> Build -> Artifact -> Deploy -> Validate -> Operate -> Recover

- Traceability links every runtime version back to its approved source change
- Automation standardizes repeatable deployment across different module types
- Validation covers infrastructure health plus module-level functional behavior
- Operational evidence supports troubleshooting, audit and future releases